Healthcare Data Analysis with Python

Data Analyst Project | Pandas + NumPy + Matplotlib + Seaborn

Developed By- Anurag Kumar Poddar

Data Analyst/Business Analyst

Project Goal

In this project, we will analyse a healthcare dataset containing patient, admission, medical condition, billing, insurance and test-result information.

We will follow a real-world data analysis process:

Load Data → Understand Data → Clean Data → Analyse Data → Visualise Results → Find Business Insights

Tools Used

- **Pandas** – data loading, cleaning and analysis
- **NumPy** – numerical calculations
- **Matplotlib** – charts
- **Seaborn** – attractive statistical visualisations

> **Important:** This notebook is designed for **Anaconda + Jupyter Notebook** and uses the local CSV path provided by the user.

1. Import the Required Libraries

We start by importing the four main Python libraries used in this project.

In [40…

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# clean chart style with a white background and gridlines.
sns.set_theme(style="whitegrid")

# Allows Pandas to display all columns of a DataFrame instead of hiding s
pd.set_option("display.max_columns", None)
```

2. Load the Healthcare Dataset

The CSV file is stored in the Windows Downloads folder.

File path: `C:\Users\anura\Downloads\healthcare_dataset.csv`

In [41…
```
file_path = r"C:\Users\anura\Downloads\healthcare_dataset.csv"

df = pd.read_csv(file_path)
```

3. First Look at the Data

Let's see the first few records and understand what kind of information is available.

In [42…
```
df.head()
```

Out[42]:

	Name	Age	Gender	Blood Type	Medical Condition	Date of Admission	Doctor	Hospital	Insurance Provider	
0	Bobby JacksOn	30	Male	B-	Cancer	31-01-2024	Matthew Smith	Sons and Miller	Blue Cross	188
1	LesLie TErRy	62	Male	A+	Obesity	20-08-2019	Samantha Davies	Kim Inc	Medicare	336
2	DaNnY sMitH	76	Female	A-	Obesity	22-09-2022	Tiffany Mitchell	Cook PLC	Aetna	279
3	andrEw waTtS	28	Female	O+	Diabetes	18-11-2020	Kevin Wells	Hernandez Rogers and Vang,	Medicare	379
4	adrIENNE bEll	43	Female	AB+	Cancer	19-09-2022	Kathleen Hanna	White-White	Aetna	142

In [43…
```
print("Rows and columns:", df.shape)

print("\nColumn names:")     #column labels
print(df.columns.tolist())   #displays all the column names
```

```
Rows and columns: (55500, 15)

Column names:
['Name', 'Age', 'Gender', 'Blood Type', 'Medical Condition', 'Date of Adm
ission', 'Doctor', 'Hospital', 'Insurance Provider', 'Billing Amount', 'R
oom Number', 'Admission Type', 'Discharge Date', 'Medication', 'Test Resu
lts']
```

4. Understand the Data Types

In [44…
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 55500 entries, 0 to 55499
Data columns (total 15 columns):
 #   Column              Non-Null Count  Dtype  
---  ------              --------------  -----  
 0   Name                55500 non-null  object 
 1   Age                 55500 non-null  int64  
 2   Gender              55500 non-null  object 
 3   Blood Type          55500 non-null  object 
 4   Medical Condition   55500 non-null  object 
 5   Date of Admission   55500 non-null  object 
 6   Doctor              55500 non-null  object 
 7   Hospital            55500 non-null  object 
 8   Insurance Provider  55500 non-null  object 
 9   Billing Amount      55500 non-null  float64
 10  Room Number         55500 non-null  int64  
 11  Admission Type      55500 non-null  object 
 12  Discharge Date      55500 non-null  object 
 13  Medication          55500 non-null  object 
 14  Test Results        55500 non-null  object 
dtypes: float64(1), int64(2), object(12)
memory usage: 6.4+ MB
```

5. Check Missing Values

Missing values can affect calculations and visualisations, so we should check them before analysis.

In [45…
```
missing_values = df.isnull().sum()

print("Missing values in each column:")
print(missing_values)

print("\nTotal missing values:", df.isnull().sum().sum())
```

```
Missing values in each column:
Name                  0
Age                   0
Gender                0
Blood Type            0
Medical Condition     0
Date of Admission     0
Doctor                0
Hospital              0
Insurance Provider    0
Billing Amount        0
Room Number           0
Admission Type        0
Discharge Date        0
Medication            0
Test Results          0
dtype: int64

Total missing values: 0
```

6. Check Duplicate Records

Duplicate records can give incorrect results. We will identify them and remove them during cleaning.

In [7]:
```
duplicate_count = df.duplicated().sum()

print("Number of duplicate rows:", duplicate_count)
```

```
Number of duplicate rows: 534
```

7. Statistical Summary

`describe()` gives us useful information about numerical columns such as age, billing amount and room number.

In [8]:
```
df.describe()
```

Out[8]:

	Age	Billing Amount	Room Number
count	55500.000000	55500.000000	55500.000000
mean	51.539459	25539.316097	301.134829
std	19.602454	14211.454431	115.243069
min	13.000000	-2008.492140	101.000000
25%	35.000000	13241.224655	202.000000
50%	52.000000	25538.069380	302.000000
75%	68.000000	37820.508432	401.000000
max	89.000000	52764.276740	500.000000

8. Data Cleaning

Now we will clean the dataset.

Main cleaning tasks:

1. Remove duplicate records
2. Convert admission and discharge dates
3. Check invalid billing amounts
4. Create useful calculated columns

8.1 Remove Duplicate Rows

In [46…

```
df = df.drop_duplicates().copy()

print("Dataset shape after removing duplicates:", df.shape)
```

```
Dataset shape after removing duplicates: (54966, 15)
```

8.2 Convert Date Columns

The admission and discharge dates are currently stored as text. We will convert them into proper datetime values.

In [47…

```
df["Date of Admission"] = pd.to_datetime(
    df["Date of Admission"],
    dayfirst=True,           #day before the month when the date is ambig
    errors="coerce"          #If Pandas encounters an invalid or unrecogn
)
```

```
df["Discharge Date"] = pd.to_datetime(
    df["Discharge Date"],
    dayfirst=True,
    errors="coerce"
)

print(df[["Date of Admission", "Discharge Date"]].head())
```

```
  Date of Admission Discharge Date
0        2024-01-31     2024-02-02
1        2019-08-20     2019-08-26
2        2022-09-22     2022-10-07
3        2020-11-18     2020-12-18
4        2022-09-19     2022-10-09
```

8.3 Check Billing Amounts

Let's check whether there are negative billing amounts. A negative hospital bill is normally not useful for this type of patient billing analysis, so we will treat those values as invalid.

In [48…
```
negative_bills = (df["Billing Amount"] < 0).sum()

print("Negative billing records:", negative_bills)
```

```
Negative billing records: 106
```

8.4 Fix Invalid Billing Values

we will replace negative billing values with the **median positive billing amount**.

Using the median reduces the effect of unusually high or low bills.

In [49…
```
valid_bills = df.loc[df["Billing Amount"] >= 0, "Billing Amount"]

median_bill = valid_bills.median()

df.loc[df["Billing Amount"] < 0, "Billing Amount"] = median_bill

print("Median valid billing amount:", round(median_bill, 2))
print("Negative bills remaining:", (df["Billing Amount"] < 0).sum())
```

```
Median valid billing amount: 25593.87
Negative bills remaining: 0
```

8.5 Create Length of Stay

-an important healthcare metric.

Length of Stay = Discharge Date - Admission Date

In [50…
```
df["Length of Stay"] = (
    df["Discharge Date"] - df["Date of Admission"]
).dt.days

# Check the new column
df[["Date of Admission", "Discharge Date", "Length of Stay"]].head()
```

Out[50]:

	Date of Admission	Discharge Date	Length of Stay
0	2024-01-31	2024-02-02	2
1	2019-08-20	2019-08-26	6
2	2022-09-22	2022-10-07	15
3	2020-11-18	2020-12-18	30
4	2022-09-19	2022-10-09	20

8.6 Create Age Groups

Age groups make it easier to compare patient populations.

In [51…
```
df["Age Group"] = pd.cut(
    df["Age"],
    bins=[0, 18, 35, 50, 65, 100],
    labels=["0-18", "19-35", "36-50", "51-65", "66+"],
    include_lowest=True
)

df[["Age", "Age Group"]].head()
```

Out[51]:

	Age	Age Group
0	30	19-35
1	62	51-65
2	76	66+
3	28	19-35
4	43	36-50

9. Check the Cleaned Dataset

Let's verify the important cleaning steps.

In [15…
```
print("Final rows:", len(df))
print("Final columns:", len(df.columns))
print("Missing values:", df.isnull().sum().sum())
print("Duplicate rows:", df.duplicated().sum())

print("\nFinal columns:")
print(df.columns.tolist())
```

```
Final rows: 54966
Final columns: 17
Missing values: 0
Duplicate rows: 0

Final columns:
['Name', 'Age', 'Gender', 'Blood Type', 'Medical Condition', 'Date of Adm
ission', 'Doctor', 'Hospital', 'Insurance Provider', 'Billing Amount', 'R
oom Number', 'Admission Type', 'Discharge Date', 'Medication', 'Test Resu
lts', 'Length of Stay', 'Age Group']
```

10. Exploratory Data Analysis

Now the dataset is ready for analysis.

- What are the most common medical conditions?
- How are patients distributed by gender?
- Which admission type is most common?
- What is the average billing amount?
- Which medical conditions have higher bills?
- How long do patients usually stay?
- What are the common test results?

10.1 Basic Patient Statistics

In [52…
```
print("Total patients:", len(df))
print("Average age:", round(df["Age"].mean(), 2))
print("Minimum age:", df["Age"].min())
print("Maximum age:", df["Age"].max())
print("Average billing amount:", round(df["Billing Amount"].mean(), 2))
print("Median billing amount:", round(df["Billing Amount"].median(), 2))
print("Average length of stay:", round(df["Length of Stay"].mean(), 2), "
```

```
Total patients: 54966
Average age: 51.54
Minimum age: 13
Maximum age: 89
Average billing amount: 25594.63
Median billing amount: 25593.87
Average length of stay: 15.5 days
```

10.2 Most Common Medical Conditions

In [53…
```
condition_counts = df["Medical Condition"].value_counts()

print(condition_counts)
```

```
Medical Condition
Arthritis       9218
Diabetes        9216
Hypertension    9151
Obesity         9146
Cancer          9140
Asthma          9095
Name: count, dtype: int64
```

10.3 Average Billing by Medical Condition

This helps us compare the average billing associated with different medical conditions.

In [54…
```
condition_billing = (
    df.groupby("Medical Condition")["Billing Amount"]
    .mean()
    .sort_values(ascending=False)
)

print(condition_billing.round(2))
```

```
Medical Condition
Obesity         25858.66
Diabetes        25714.08
Asthma          25685.20
Hypertension    25559.92
Arthritis       25542.96
Cancer          25206.73
Name: Billing Amount, dtype: float64
```

10.4 Admission Type Analysis

In [55…
```
admission_counts = df["Admission Type"].value_counts()
```

```
print(admission_counts)
```

```
Admission Type
Elective     18473
Urgent       18391
Emergency    18102
Name: count, dtype: int64
```

10.5 Test Result Analysis

In [56…
```
test_results = df["Test Results"].value_counts()

print(test_results)
```

```
Test Results
Abnormal        18437
Normal          18331
Inconclusive    18198
Name: count, dtype: int64
```

10.6 Gender Analysis

In [21…
```
gender_counts = df["Gender"].value_counts()

print(gender_counts)
```

```
Gender
Male      27496
Female    27470
Name: count, dtype: int64
```

10.7 Insurance Provider Analysis

In [57…
```
insurance_counts = df["Insurance Provider"].value_counts()

print(insurance_counts)
```

```
Insurance Provider
Cigna               11139
Medicare            11039
UnitedHealthcare    11014
Blue Cross          10952
Aetna               10822
Name: count, dtype: int64
```

11. NumPy Analysis

We will use NumPy for numerical calculations. -Mean -Median -SD -Minimum

-Maximum

In [23…
```
billing_values = df["Billing Amount"].to_numpy()

print("Billing mean:", np.mean(billing_values))
print("Billing median:", np.median(billing_values))
print("Billing standard deviation:", np.std(billing_values))
print("Minimum billing:", np.min(billing_values))
print("Maximum billing:", np.max(billing_values))
```

```
Billing mean: 25594.63216986607
Billing median: 25593.872994999998
Billing standard deviation: 14162.062537801829
Minimum billing: 9.238787497
Maximum billing: 52764.27674
```

12. Create a Simple Summary Table

A grouped summary is often useful for reporting to managers or business teams.

In [59…
```
summary_table = (
    df.groupby("Medical Condition")
    .agg(
        Patients=("Name", "count"),
        Average_Age=("Age", "mean"),
        Average_Billing=("Billing Amount", "mean"),
        Average_Stay=("Length of Stay", "mean")
    )
    .sort_values("Patients", ascending=False)   #highest to lowest.
)

summary_table.round(2)
```

Out[59]:

Medical Condition	Patients	Average_Age	Average_Billing	Average_Stay
Arthritis	9218	51.54	25542.96	15.50
Diabetes	9216	51.58	25714.08	15.43
Hypertension	9151	51.72	25559.92	15.44
Obesity	9146	51.23	25858.66	15.45
Cancer	9140	51.54	25206.73	15.50
Asthma	9095	51.60	25685.20	15.68

13. Data Visualisation

Now we will create charts using **Matplotlib and Seaborn**.

The charts help us understand patterns much faster than looking at raw tables.

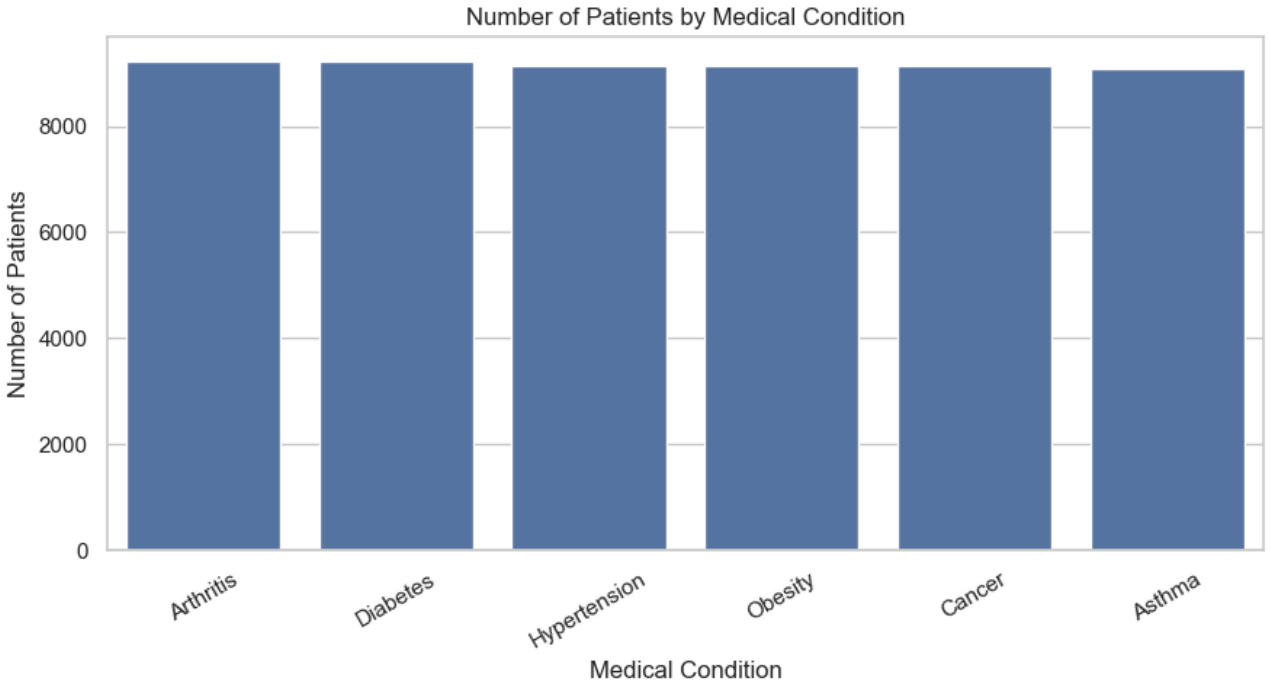
13.1 Number of Patients by Medical Condition

In [25…

```
plt.figure(figsize=(9, 5))

sns.countplot(
    data=df,
    x="Medical Condition",
    order=df["Medical Condition"].value_counts().index
)

plt.title("Number of Patients by Medical Condition")
plt.xlabel("Medical Condition")
plt.ylabel("Number of Patients")
plt.xticks(rotation=30)
plt.tight_layout()
plt.show()
```



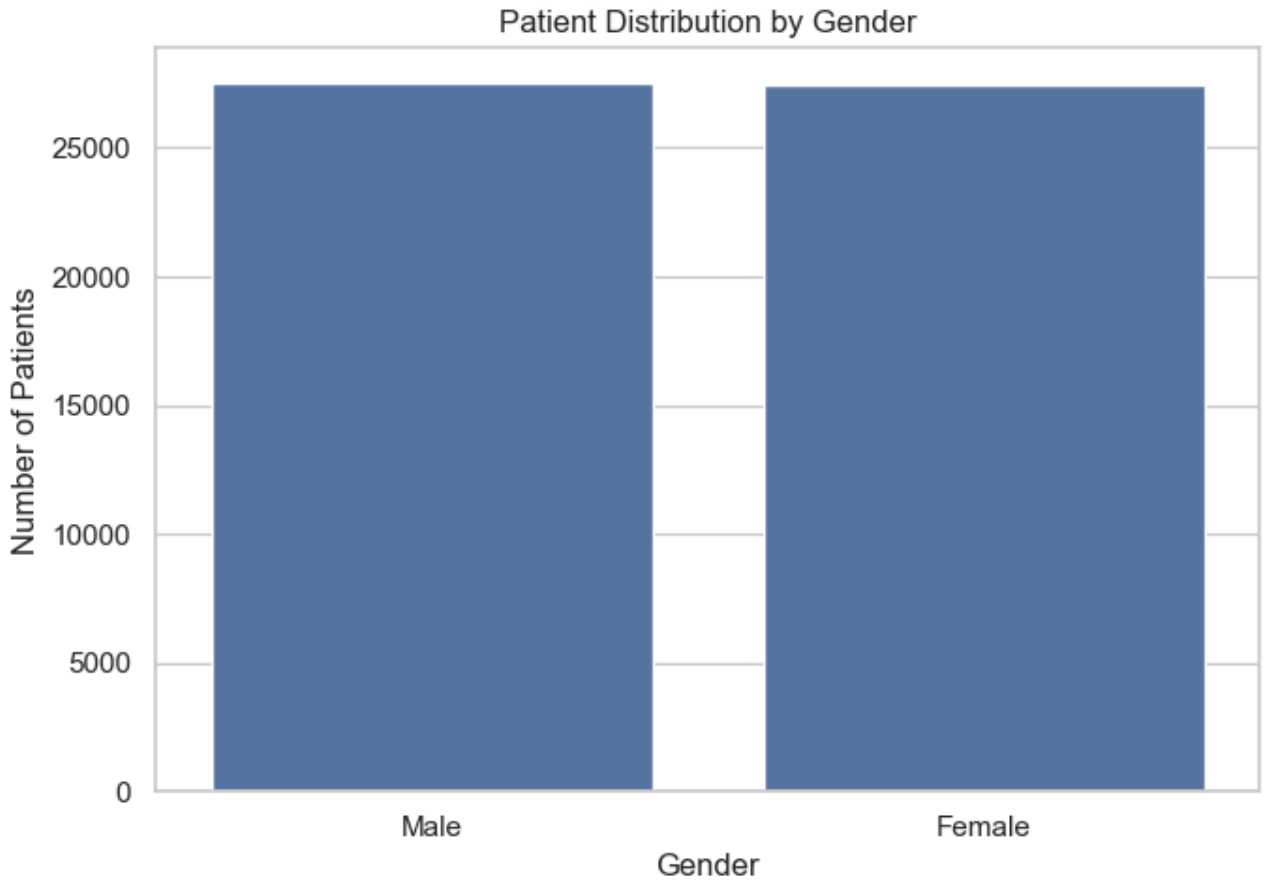
13.2 Patient Distribution by Gender

In [26…

```
plt.figure(figsize=(7, 5))

sns.countplot(
    data=df,
    x="Gender",
    order=df["Gender"].value_counts().index
)

plt.title("Patient Distribution by Gender")
plt.xlabel("Gender")
plt.ylabel("Number of Patients")
plt.tight_layout()
plt.show()
```



13.3 Admission Type

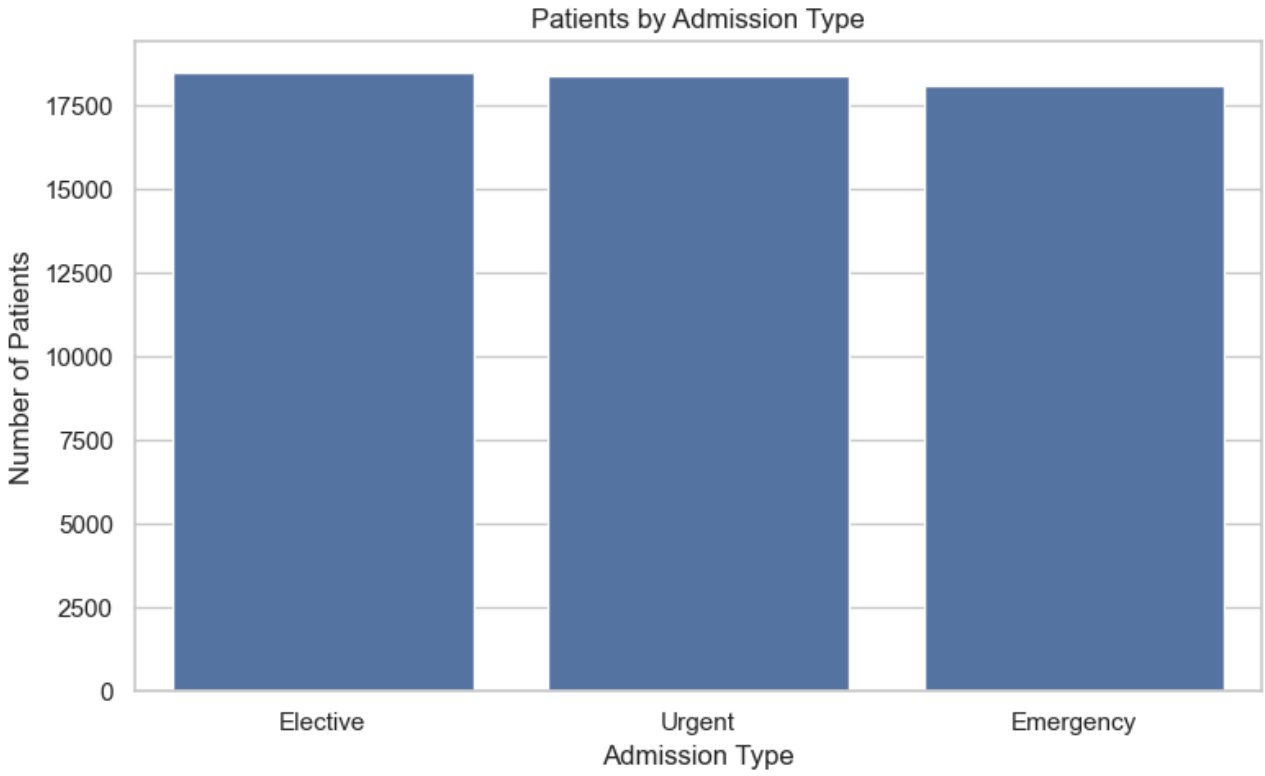
In [27…

```
plt.figure(figsize=(8, 5))

sns.countplot(
    data=df,
```

```
    x="Admission Type",
    order=df["Admission Type"].value_counts().index
)

plt.title("Patients by Admission Type")
plt.xlabel("Admission Type")
plt.ylabel("Number of Patients")
plt.tight_layout()
plt.show()
```



13.4 Age Distribution

A histogram helps us understand the distribution of patient ages.

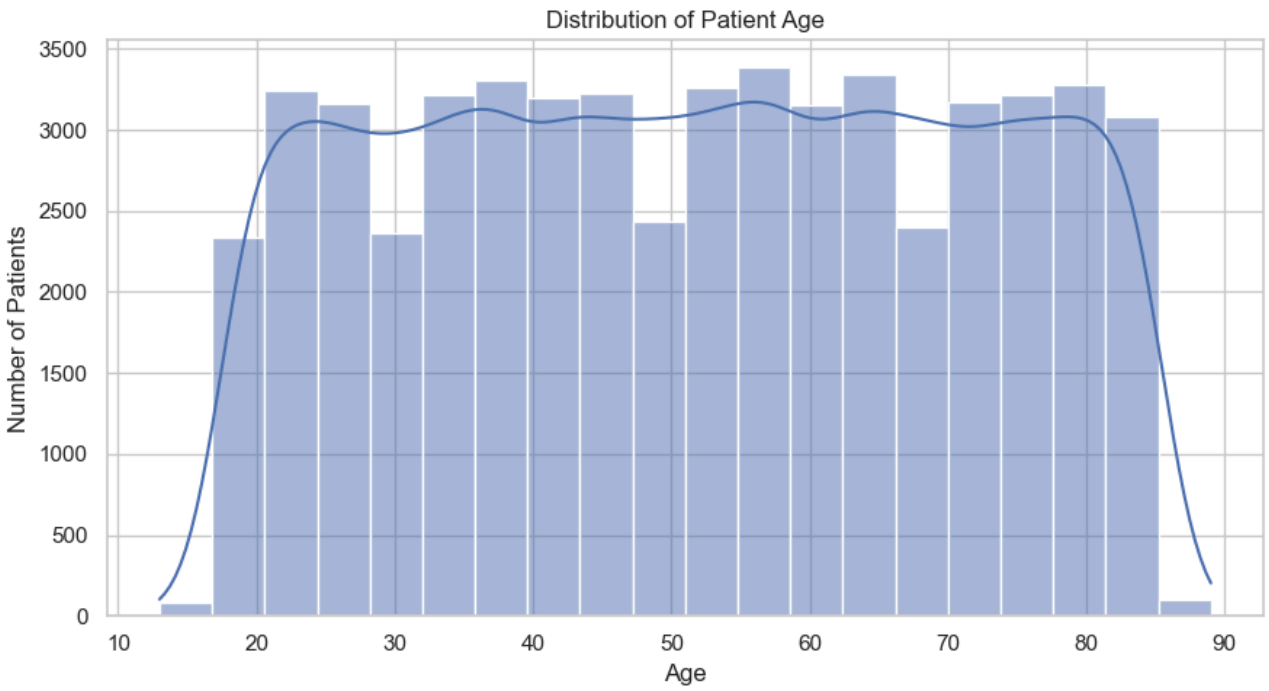
In [28…

```
plt.figure(figsize=(9, 5))

sns.histplot(
    data=df,
    x="Age",
    bins=20,
    kde=True
)

plt.title("Distribution of Patient Age")
plt.xlabel("Age")
plt.ylabel("Number of Patients")
plt.tight_layout()
```

```
plt.show()
```



13.5 Billing Amount by Medical Condition

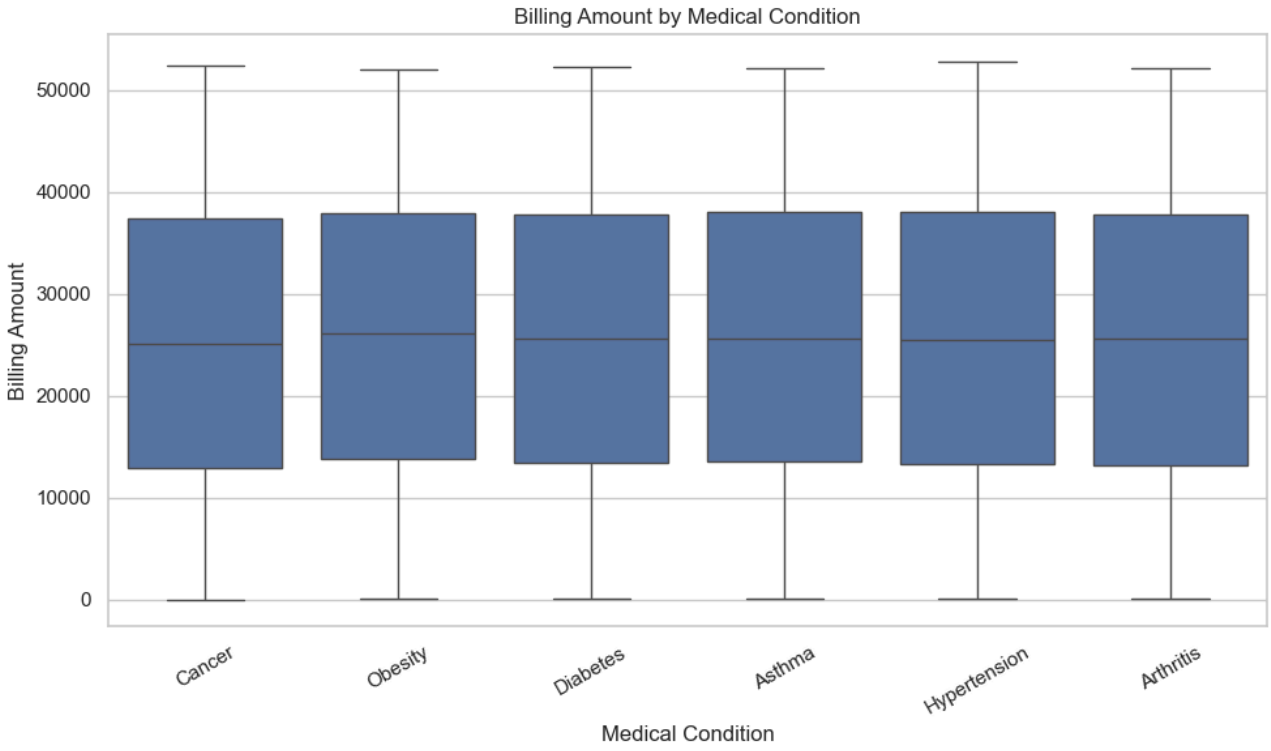
A box plot helps us compare the spread of billing amounts.

In [60…

```
plt.figure(figsize=(10, 6))

sns.boxplot(
    data=df,
    x="Medical Condition",
    y="Billing Amount"
)

plt.title("Billing Amount by Medical Condition")
plt.xlabel("Medical Condition")
plt.ylabel("Billing Amount")
plt.xticks(rotation=30)
plt.tight_layout()
plt.show()
```



13.6 Average Billing by Medical Condition

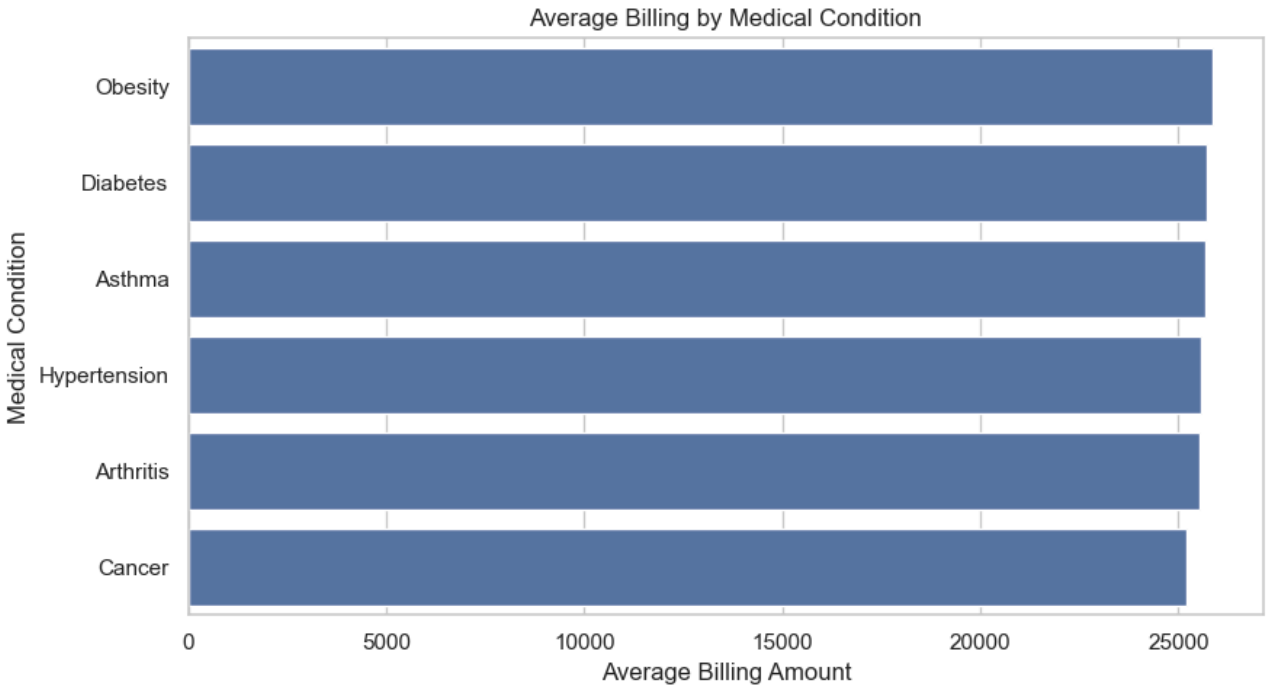
In [30…

```
plt.figure(figsize=(9, 5))

billing_plot = (
    df.groupby("Medical Condition")["Billing Amount"]
    .mean()
    .sort_values(ascending=False)
)

sns.barplot(
    x=billing_plot.values,
    y=billing_plot.index
)

plt.title("Average Billing by Medical Condition")
plt.xlabel("Average Billing Amount")
plt.ylabel("Medical Condition")
plt.tight_layout()
plt.show()
```



13.7 Test Results by Gender

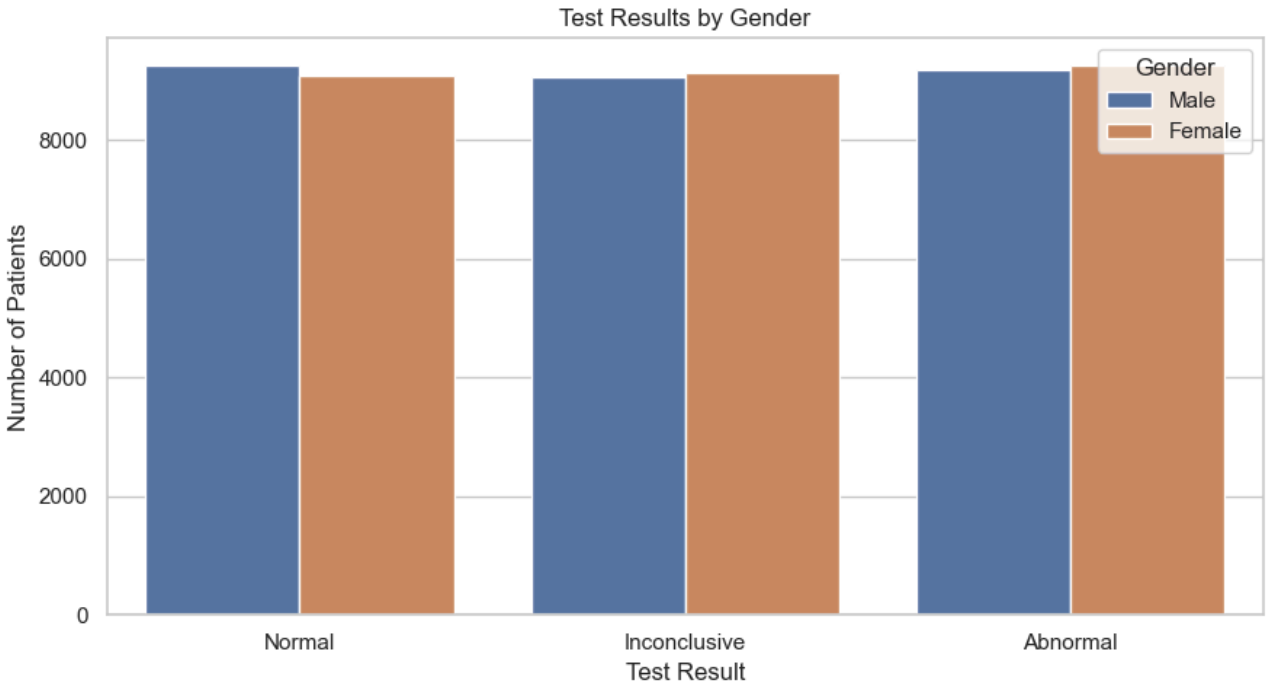
A grouped count plot allows us to compare test-result categories across genders.

In [31…

```
plt.figure(figsize=(9, 5))

sns.countplot(
    data=df,
    x="Test Results",
    hue="Gender"
)

plt.title("Test Results by Gender")
plt.xlabel("Test Result")
plt.ylabel("Number of Patients")
plt.tight_layout()
plt.show()
```



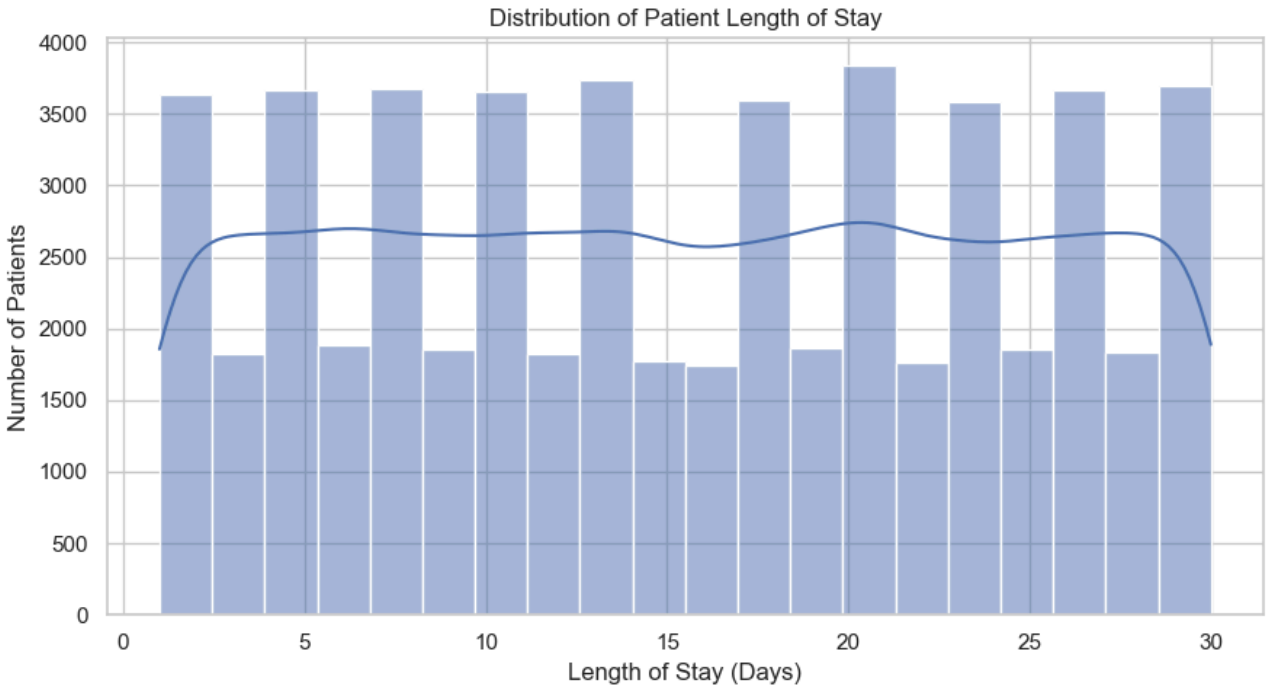
13.8 Length of Stay Distribution

In [32…

```
plt.figure(figsize=(9, 5))

sns.histplot(
    data=df,
    x="Length of Stay",
    bins=20,
    kde=True
)

plt.title("Distribution of Patient Length of Stay")
plt.xlabel("Length of Stay (Days)")
plt.ylabel("Number of Patients")
plt.tight_layout()
plt.show()
```



13.9 Monthly Patient Admissions

We create a month column and look at how the number of admissions changes over time.

In [33…
```
df["Admission Month"] = df["Date of Admission"].dt.to_period("M").astype(

monthly_admissions = (
    df.groupby("Admission Month")
    .size()
    .reset_index(name="Patients")
)

monthly_admissions.head()
```

Out[33]:

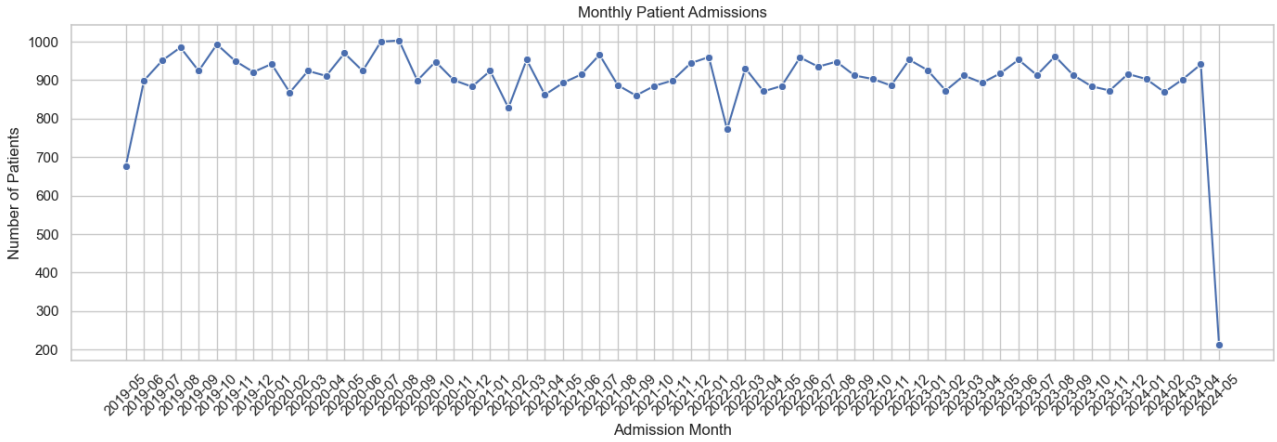
	Admission Month	Patients
0	2019-05	677
1	2019-06	899
2	2019-07	951
3	2019-08	985
4	2019-09	924

In [61…
```
plt.figure(figsize=(14, 5))

sns.lineplot(
```

```
    data=monthly_admissions,
    x="Admission Month",
    y="Patients",
    marker="o"
)

plt.title("Monthly Patient Admissions")
plt.xlabel("Admission Month")
plt.ylabel("Number of Patients")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



14. Correlation Analysis

Correlation helps us understand whether numerical variables move together.it calculates the relationship between numerical columns and then displays those relationships in a colourful chart - correlation heatmap.

We will look at:

- Age
- Billing Amount
- Room Number
- Length of Stay

In [62…

```
numeric_columns = [
    "Age",
    "Billing Amount",
    "Room Number",
    "Length of Stay"
]

correlation = df[numeric_columns].corr()
```

```
print(correlation.round(2))     #values to two decimal places.
```

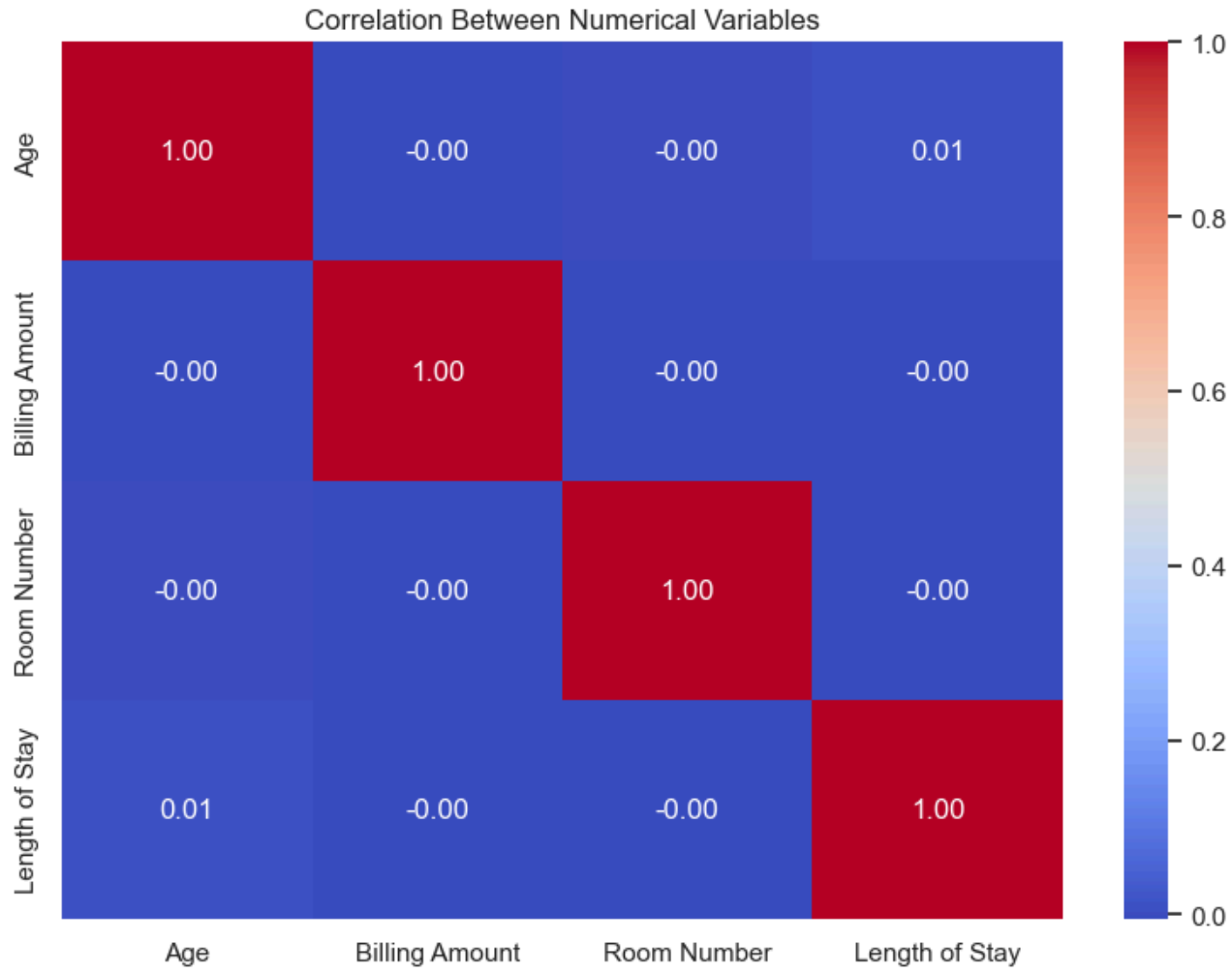
```
                 Age  Billing Amount  Room Number  Length of Stay
Age             1.00            -0.0         -0.0            0.01
Billing Amount -0.00             1.0         -0.0           -0.00
Room Number    -0.00            -0.0          1.0           -0.00
Length of Stay  0.01            -0.0         -0.0            1.00
```

In [63…

```
plt.figure(figsize=(8, 6))

sns.heatmap(
    correlation,
    annot=True,
    cmap="coolwarm",     #uses contrasting cool and warm colours to disti
    fmt=".2f"            #two decimal places
)

plt.title("Correlation Between Numerical Variables")
plt.tight_layout()
plt.show()
```



15. Top Findings

The following code automatically generates a few important findings from the cleaned dataset.

In [64…

```
top_condition = df["Medical Condition"].value_counts().idxmax()
top_condition_count = df["Medical Condition"].value_counts().max()

highest_billing_condition = (
    df.groupby("Medical Condition")["Billing Amount"]
    .mean()
    .idxmax()
)

highest_billing_value = (
    df.groupby("Medical Condition")["Billing Amount"]
    .mean()
    .max()
)

most_common_admission = df["Admission Type"].value_counts().idxmax()
most_common_test = df["Test Results"].value_counts().idxmax()

print("KEY FINDINGS")
print("-" * 50)

print(f"1. Most common medical condition: {top_condition}")
print(f"   Number of patients: {top_condition_count}")

print(f"\n2. Highest average billing condition: {highest_billing_conditio
print(f"   Average billing: {highest_billing_value:.2f}")

print(f"\n3. Most common admission type: {most_common_admission}")

print(f"\n4. Most common test result: {most_common_test}")

print(f"\n5. Overall average patient age: {df['Age'].mean():.2f} years")

print(f"\n6. Overall average billing: {df['Billing Amount'].mean():.2f}")

print(f"\n7. Average length of stay: {df['Length of Stay'].mean():.2f} da
```

```
KEY FINDINGS
--------------------------------------------------
1. Most common medical condition: Arthritis
   Number of patients: 9218

2. Highest average billing condition: Obesity
   Average billing: 25858.66

3. Most common admission type: Elective

4. Most common test result: Abnormal

5. Overall average patient age: 51.54 years

6. Overall average billing: 25594.63

7. Average length of stay: 15.50 days
```

16. Business Insights

1. Patient Demographics

The dataset contains 54,966 records after duplicate removal. The average patient age is 51.54 years, with ages ranging from 13 to 89. Male and female patient counts are almost equal, indicating a balanced gender distribution in the analysed records.

1. Medical Condition Analysis

Arthritis is the most frequently recorded medical condition, with 9,218 records, closely followed by Diabetes with 9,216 records. Patient counts across all six medical conditions are relatively similar, suggesting that the dataset does not show a single condition overwhelmingly dominating the recorded patient population.

1. Billing and Financial Analysis

The overall average billing amount is 25,594.63. Among the analysed medical conditions, Obesity has the highest average billing amount at 25,858.66, while Cancer has the lowest at 25,206.73. This difference can help identify areas for further investigation into treatment costs, resource utilisation, and billing patterns.

1. Hospital Admission Patterns

Elective admissions are the most common, with 18,473 records, followed by Urgent admissions (18,391) and Emergency admissions (18,102). The relatively close counts suggest that all three admission categories contribute substantially to the dataset.

1. Length of Hospital Stay

The average length of stay is 15.50 days. Average stays are broadly similar across medical conditions, although Asthma has the highest average at approximately 15.68 days. This metric can support further investigation into bed utilisation, discharge planning, and operational efficiency.

1. Test Result Analysis

Abnormal is the most frequently recorded test-result category, with 18,437 records, followed by Normal and Inconclusive results. This distribution may help guide further analysis of test outcomes by medical condition, age group, and gender. However, these counts alone do not establish the severity of patient conditions.

1. Data Quality and Reliability

The initial dataset contained 534 duplicate records and 106 negative billing values. Duplicate records were removed, and negative billing values were replaced with the median of valid non-negative billing amounts. Admission and discharge dates were converted into datetime format, and additional analytical columns were created for age groups and length of stay. These steps improved the dataset's consistency for further analysis.

1. Correlation Analysis

The correlation analysis showed almost no linear relationship between age, billing amount, and length of stay in this dataset. This suggests that these variables do not have strong linear associations in the analysed records. Further analysis would be required to identify other factors influencing billing and hospital stay duration.

17. Save the Cleaned Dataset

After cleaning, we can save a new CSV file so the original dataset remains unchanged.

The new file will be created in the same Downloads folder.

In [38…
```
output_path = r"C:\Users\anura\Downloads\healthcare_dataset_cleaned.csv"

df.to_csv(output_path, index=False)

print("Cleaned dataset saved successfully!")
print(output_path)
```

```
Cleaned dataset saved successfully!
C:\Users\anura\Downloads\healthcare_dataset_cleaned.csv
```

18. Project Conclusion

This project demonstrates an end-to-end healthcare data analysis workflow using Python. Starting with raw healthcare records, I performed data inspection, data cleaning, feature engineering, exploratory data analysis, statistical calculations, correlation analysis, and data visualisation using Pandas, NumPy, Matplotlib, and Seaborn.

The analysis highlighted useful patterns in patient demographics, medical conditions, billing amounts, admission types, test results, and hospital stays. It also demonstrated the importance of handling duplicate records and invalid billing values before generating reports and insights.

Overall, this project strengthened my practical understanding of data cleaning, exploratory data analysis (EDA), statistical analysis, data visualisation, and business insight generation. The findings can support further investigation into healthcare costs and operational patterns, while more detailed clinical and operational data would be needed to make recommendations about patient outcomes or hospital performance.

Skills Demonstrated

Python | Pandas | NumPy | Matplotlib | Seaborn | Data Cleaning | EDA | Data Visualisation | Healthcare Analytics

---

Project Developed By - Anurag Kumar Poddar

Healthcare Patient Data Analysis using Python

A end-to-end data analytics project demonstrating how raw healthcare data

can be cleaned, analysed and converted into meaningful business insights using Python.

In [ ]: